

Yes.  **Server.**

And now let's make the reasoning precise, because this is exactly how you'd explain the architecture in an interview.

Our current architecture

Browser

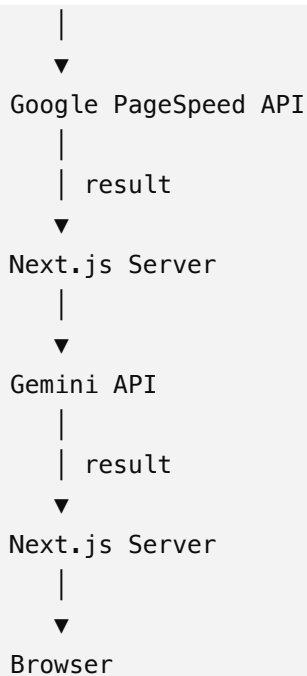
|

| "Analyze this URL"

▼

Next.js Server





Why server?

- 🗝️ API keys/secrets stay private
- 🧠 PageSpeed → Gemini orchestration happens server-side
- 🛡️ Validation/error handling/rate limiting can happen server-side
- 📦 Browser doesn't need to contain the external API logic

And **PageSpeed → Gemini remains sequential** because Gemini needs PageSpeed's result.

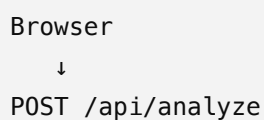
Now here's the next important decision

We have two common ways to initiate this from the UI:

Option A — Server Component



Option B — Route Handler



↓
Next.js Route Handler
↓
PageSpeed
↓
Gemini

For your "user enters a URL → clicks Analyze" interaction, which one do you think makes more sense?

A or B — and why?



b

Yes — B is the better fit. 🎯

Because the user is performing an **interactive action**:

User types URL
↓
Clicks "Analyze"
↓
send data to server



A **Route Handler** gives us a clear server endpoint:

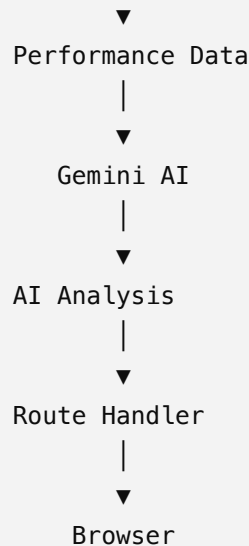
POST /api/analyze



So the architecture becomes:

BROWSER
|
POST /api/analyze
{ url: "..."}
|
▼
NEXT.JS SERVER
|
Route Handler
|
▼
Google PageSpeed
|





Why not a Server Component?

A Server Component is primarily about **rendering UI on the server**.

A Route Handler is appropriate when the browser needs to **send an HTTP request to your server and receive data back**.

So:

Server Component
→ "Render this UI using server-side data."



Route Handler
→ "Here's an API endpoint my client can call."

For your Analyze button, the second model fits naturally.

One thing to watch out for 🙄

If your Route Handler does:

PageSpeed → Gemini



and waits for **both** before returning:

```
return Response.json({
  performance,
  analysis
});
```



then the browser gets **nothing until Gemini finishes**.

That would undermine the progressive UI we just designed.

So now we have an interesting problem:

How do we keep PageSpeed → Gemini sequential, but still show the Performance results before Gemini finishes?

That's our next architecture decision — and it's where **streaming, Suspense, and the Route Handler approach** all collide. 🔥



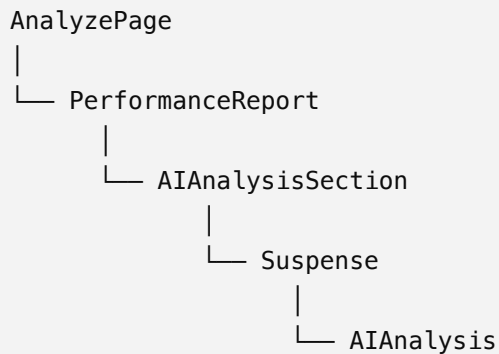
Yes — **this is exactly the confusing part**, and I want to correct the structure from my previous answer.

If **AI Analysis depends on the PageSpeed result**, and you want:

1. PageSpeed result to appear first
2. Gemini to start only after PageSpeed finishes
3. AI Analysis to be behind Suspense
4. AI Analysis to stream in later

then the clean component structure is to have a **parent Server Component orchestrate the data**, while the **AI component itself suspends**.

The structure



But the crucial part is **where the PageSpeed result is fetched**.

We do this:

</> TypeScript

```
async function PerformanceReport({ url }) {
  const performance = await getPageSpeedData(url);

  return (
    <>
      /* This can render immediately */
      <PerformanceResults data={performance} />

      /* This starts AFTER performance is available */
      <Suspense fallback=<AIAalysisSkeleton />>
        <AIAalysis performance={performance} />
      </Suspense>
    </>
  );
}
```

And:

</> TypeScript

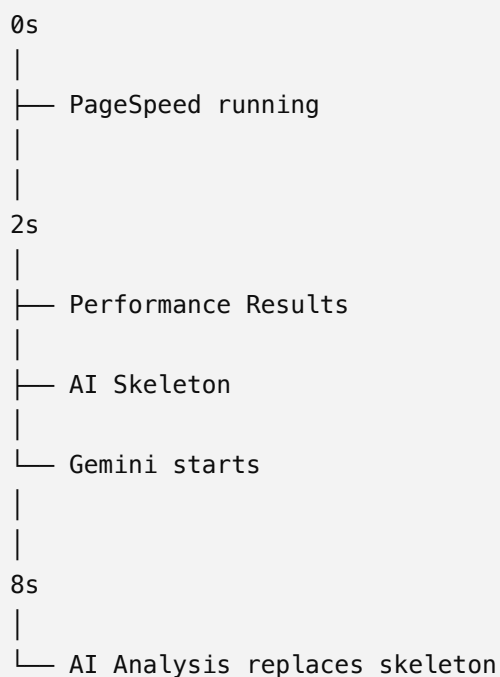
```
async function AIAalysis({ performance }) {
  const analysis = await getGeminiAnalysis(performance);
```

```
return <AIAnalysisUI analysis={analysis} />;  
}
```

Now look at the timeline



So:



That's the behavior you're looking for. 🎯

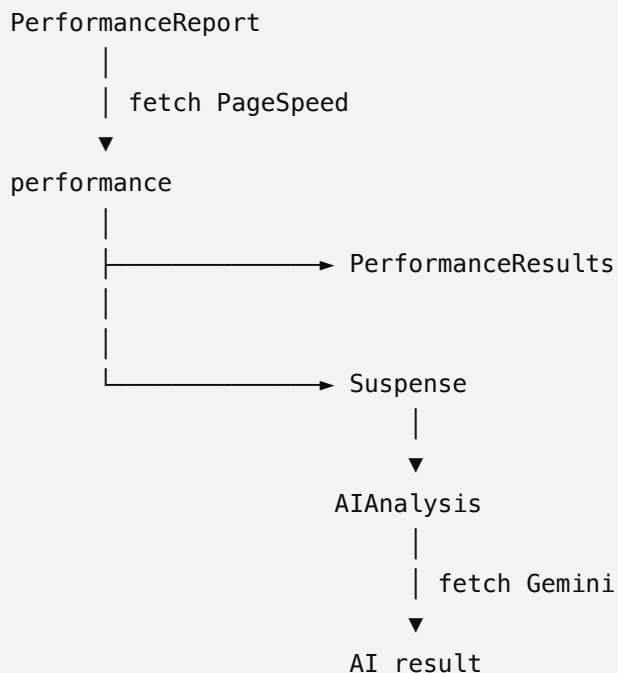
But where exactly is Suspense?

It's around the component that performs the slow Gemini operation:

</> TypeScript

```
async function PerformanceReport({ url }) {  
  const performance = await getPageSpeedData(url);  
  
  return (  
    <>  
      <PerformanceResults data={performance} />  
  
      <Suspense fallback={<AIAnalysisSkeleton />>  
        <AIAnalysis performance={performance} />  
      </Suspense>  
    </>  
  );  
}
```

So the important relationship is:



And this is why the parent matters

You were originally saying:

"Put Gemini inside the Performance component."

That instinct was actually good. 😊

The reason is that the parent has the PageSpeed result:

```
</> TypeScript
const performance = await getPageSpeedData(url);
```

Therefore it can pass that result to Gemini:

```
</> TypeScript
<AIAnalysis performance={performance} />
```

So the dependency is naturally represented:

```
PerformanceReport
  |
  ▼
performance
  |
  ▼
AIAnalysis(performance)
```

You don't need to make `AIAnalysis` fetch PageSpeed again.

One very important thing

You might ask:

"But `PerformanceReport` itself is waiting 2 seconds before it returns JSX. Doesn't that prevent streaming?"

No — because the Suspense boundary is inside the returned tree, and once PageSpeed finishes, React can render the ready part and then encounter the suspended AI component.

So after PageSpeed finishes:

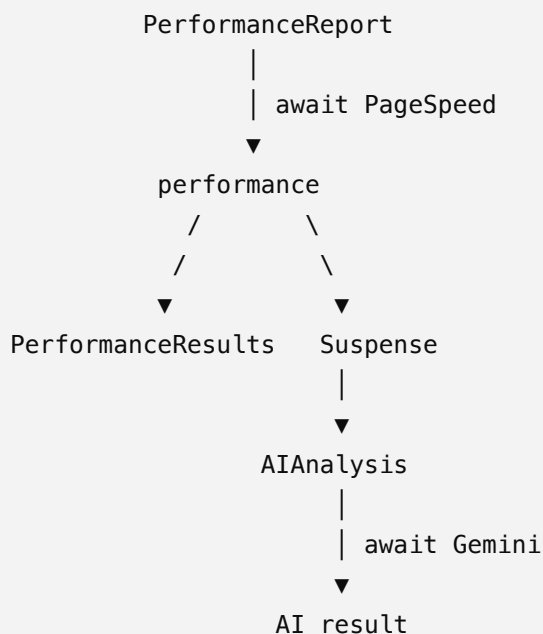
```
PerformanceReport
  |
  ├── PerformanceResults → READY
  |
  └── AIAnalysis → SUSPENDS
      |
```

▼
fallback

React can stream the ready portion plus the fallback, then later stream the completed AI section.

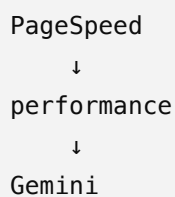
The exact mental model

This is the part I want you to remember:

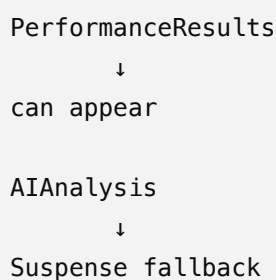


Therefore:

Data dependency



UI dependency



↓
appears later

And **that's the exact combination you were trying to describe.** ✅

One final nuance: in a real app, if you want the **PerformanceResults to stream as soon as possible while PageSpeed itself is loading**, you'd put the PageSpeed-fetching component behind its own boundary too. But for your specific requirement — **PageSpeed first, then dependent Gemini streamed afterward** — the structure above is the clean conceptual model.



in a real app, if you want the **PerformanceResults to stream as soon as possible while PageSpeed itself is loading**, you'd put the PageSpeed-fetching component behind its own boundary too...how? meaning?

Yes — and this is an important distinction. Let me show you **exactly what I meant**, because there are actually **two different levels of streaming** here. 🙌

First, what we had before

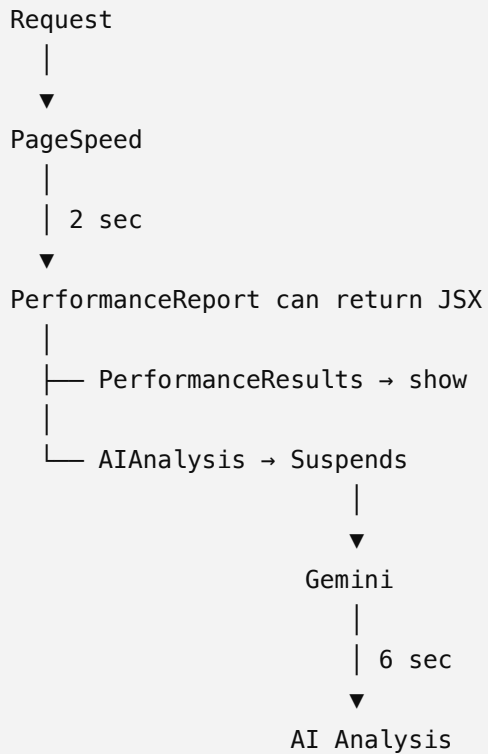
We had:

</> TypeScript



```
async function PerformanceReport({ url }) {  
  const performance = await getPageSpeedData(url); // waits here  
  
  return (  
    <>  
      <PerformanceResults data={performance} />  
  
      <Suspense fallback={<AIAnalysisSkeleton />>  
        <AIAnalysis performance={performance} />  
      </Suspense>  
    </>  
  );  
}
```

Timeline:

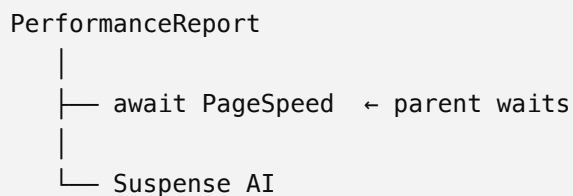


So **nothing** from `PerformanceReport` can be streamed until `PageSpeed` finishes, because the parent itself is waiting.

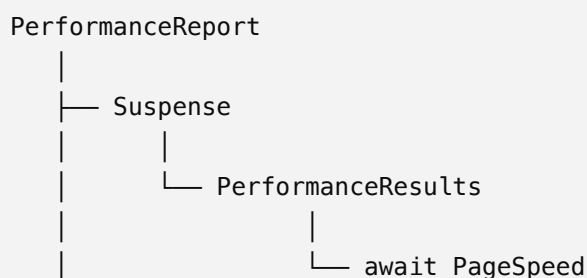
What if we want the Performance section itself to have a loading UI?

Then we move the `await` inside a child component and put that child behind `Suspense`.

Instead of:



we do:



└ ...

For example:

</> TypeScript



```
function PerformanceReport({ url }) {  
  return (  
    <  
      <Suspense fallback=<PerformanceSkeleton />>  
        <PerformanceResults url={url} />  
      </Suspense>  
  
      { /* other content */ }  
    </>  
  );  
}
```

Then:

</> TypeScript



```
async function PerformanceResults({ url }) {  
  const performance = await getPageSpeedData(url);  
  
  return <PerformanceUI data={performance} />;  
}
```

Now the parent **doesn't wait for PageSpeed**.

It immediately encounters:

</> TypeScript



```
<Suspense fallback=<PerformanceSkeleton />>
```

and can render the rest of the page.

But here's where your Gemini dependency makes things interesting

You said:

Gemini depends on PageSpeed.

Correct.

So we have:

```
PageSpeed
  ↓
performanceData
  ↓
Gemini
```



We therefore **cannot** independently do:

```
</> TypeScript

<Suspense>
  <PerformanceResults />
</Suspense>

<Suspense>
  <AIAnalysis />
</Suspense>
```



if `AIAnalysis` needs the actual `PageSpeed` result.

Because then:

```
PerformanceResults → PageSpeed
AIAnalysis → ???
```



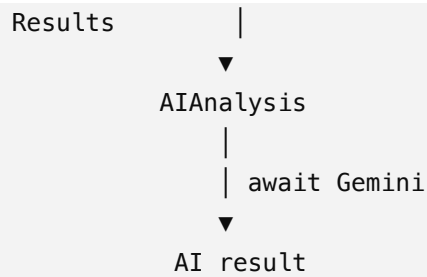
AI doesn't have the result yet.

A better real-world structure

We can make the **data dependency** itself a nested component tree:

```
AnalyzePage
├── Suspense
│   └── PerformanceSection
│       └── await PageSpeed
│           ▼
│       performanceData
│           ├── Performance
│           └── Suspense
```





Code:

</> TypeScript



```
function AnalyzePage({ url }) {
  return (
    <Suspense fallback=<PerformanceSkeleton />>
      <PerformanceSection url={url} />
    </Suspense>
  );
}
```

Then:

</> TypeScript



```
async function PerformanceSection({ url }) {
  const performance = await getPageSpeedData(url);

  return (
    <>
      <PerformanceResults data={performance} />

      <Suspense fallback=<AIAnalysisSkeleton />>
        <AIAnalysis performance={performance} />
      </Suspense>
    </>
  );
}
```

And:

</> TypeScript



```
async function AIAnalysis({ performance }) {
  const analysis = await getGeminiAnalysis(performance);

  return <AIAnalysisUI data={analysis} />;
}
```

Now watch the timeline

Suppose:

PageSpeed = 2 seconds
Gemini = 6 seconds



0 seconds

The server starts:

```
AnalyzePage
├── PerformanceSection
│   └── PageSpeed ⌚
```



Because `PerformanceSection` is suspended, React can send:

```
┌ Performance
│ [Loading performance...]
└ AI Analysis
```



2 seconds

PageSpeed finishes:

```
performanceData
  ↓
PerformanceResults
  ↓
READY
```



Then:

```
AIAnalysis
  ↓
Gemini ⌚
```



So React can stream:

```
Performance Score: 87
LCP: 1.8s
CLS: 0.04

AI Analysis
[Generating analysis...]
```



8 seconds

Gemini finishes:

```
Gemini
  ↓
AI result
```



React streams the completed boundary:

```
Performance Score: 87
LCP: 1.8s
CLS: 0.04

AI Analysis
"Your biggest issue is..."
```



So there are actually TWO Suspense boundaries

This is the key idea:

```
</> TypeScript

<Suspense fallback={<PerformanceSkeleton />}>

  <PerformanceSection>

    <PerformanceResults />

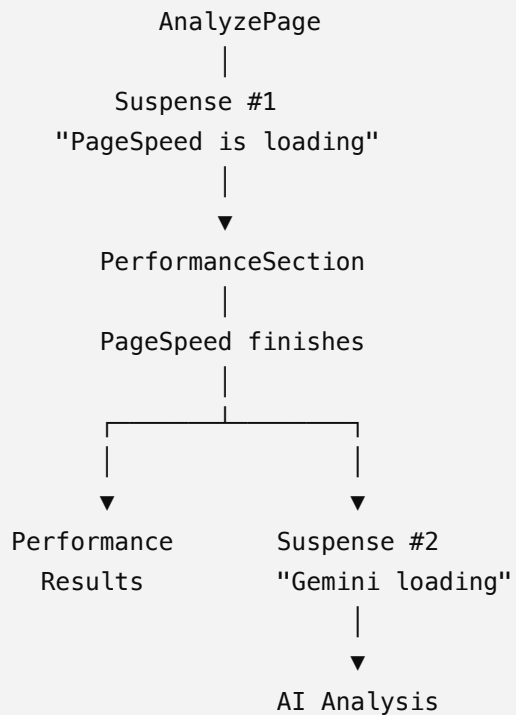
    <Suspense fallback={<AIAnalysisSkeleton />}>
      <AIAnalysis />
    </Suspense>

  </PerformanceSection>
```



`</Suspense>`

Think of them as:



Suspense #1

Controls:

"What should I show while PageSpeed is running?"

Suspense #2

Controls:

"What should I show while Gemini is running?"

And because Gemini depends on PageSpeed:

PageSpeed
↓
Gemini

they remain **sequential**.

But the UI is progressively delivered:

Performance skeleton
↓
Performance results + AI skeleton

↓
Performance results + AI results

🔥 That's what I meant by putting the PageSpeed-fetching component behind its own Suspense boundary.

And notice the beautiful rule here:

Put a Suspense boundary around the component that can suspend, not necessarily around the API call itself.

The `await` causes the component to suspend; the surrounding `<Suspense>` determines what React can show while it's suspended.



is this end of lesson 4

Yes. 🎉 Lesson 4 is now complete.

We've covered the full set of concepts we planned:

🚩 Lesson 4 — Data Fetching & Rendering Architecture

- ✓ Server-side data fetching
- ✓ Client vs Server data fetching
- ✓ Sequential fetching
- ✓ Parallel fetching
- ✓ `Promise.all`
- ✓ Caching / revalidation
- ✓ Loading UI
- ✓ Suspense
- ✓ Streaming
- ✓ Async Server Components & suspension
- ✓ HTML vs RSC payload
- ✓ Hydration
- ✓ Route Handlers / BFF
- ✓ Error handling
 - expected errors
 - unexpected errors
 - `notFound()`

- `error.tsx`
-  How all of these fit together architecturally

And most importantly, you now have the mental model:

Data dependency



Sequential / Parallel

UI waiting



Suspense

Progressive delivery



Streaming

Server-only logic



Server Components / Route Handlers

Browser interactivity

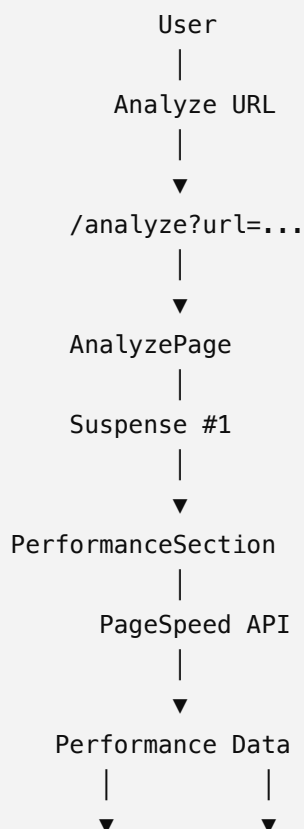


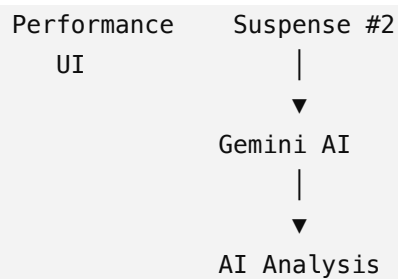
Client Components + Hydration



Your final analyzer architecture

Conceptually:





That's a solid foundation for a real Next.js application **and** for explaining the architecture in interviews.

Next: Lesson 5

When you start Lesson 5, we'll move into the next major area of Next.js — **mutations and user interactions**: forms, Server Actions, revalidation, optimistic UI, and how client interactions communicate with the server.

Lesson 4 = 🏁 COMPLETE.



ChatGPT can make mistakes. Check important info.